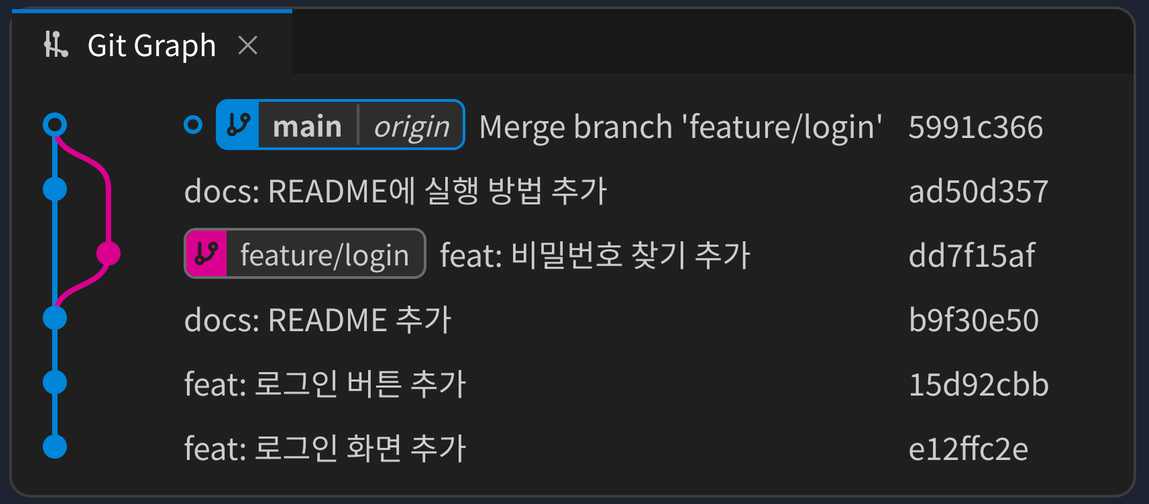
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git Graph, 눈으로 보기

VS Code에서 커밋과 브랜치를
그림으로 확인하는 법



오늘의 순서

1

설치하고 열기

확장 설치, Git Graph 여는 세 가지 방법

4–5쪽

2

화면 읽기

점, 이름표, HEAD, origin/main

6–9쪽

3

명령 치고 확인하기

commit, push, fetch, pull, checkout, merge 뒤의 그림

10–20쪽

4

쓸 때 주의, 막힐 때

명령은 터미널에서, 안 보일 때

21–22쪽

마지막 23쪽은 한 장 요약이에요.

Git Graph란?

git log --oneline을 그림으로 보여 줘요

```
$ git log --oneline
5991c36 (HEAD -> main, origin/main) Merge branch 'feature/login'
ad50d35 docs: README에 실행 방법 추가
dd7f15a (feature/login) feat: 비밀번호 찾기 추가
b9f30e5 docs: README 추가
15d92cb feat: 로그인 버튼 추가
e12ffc2 feat: 로그인 화면 추가
```

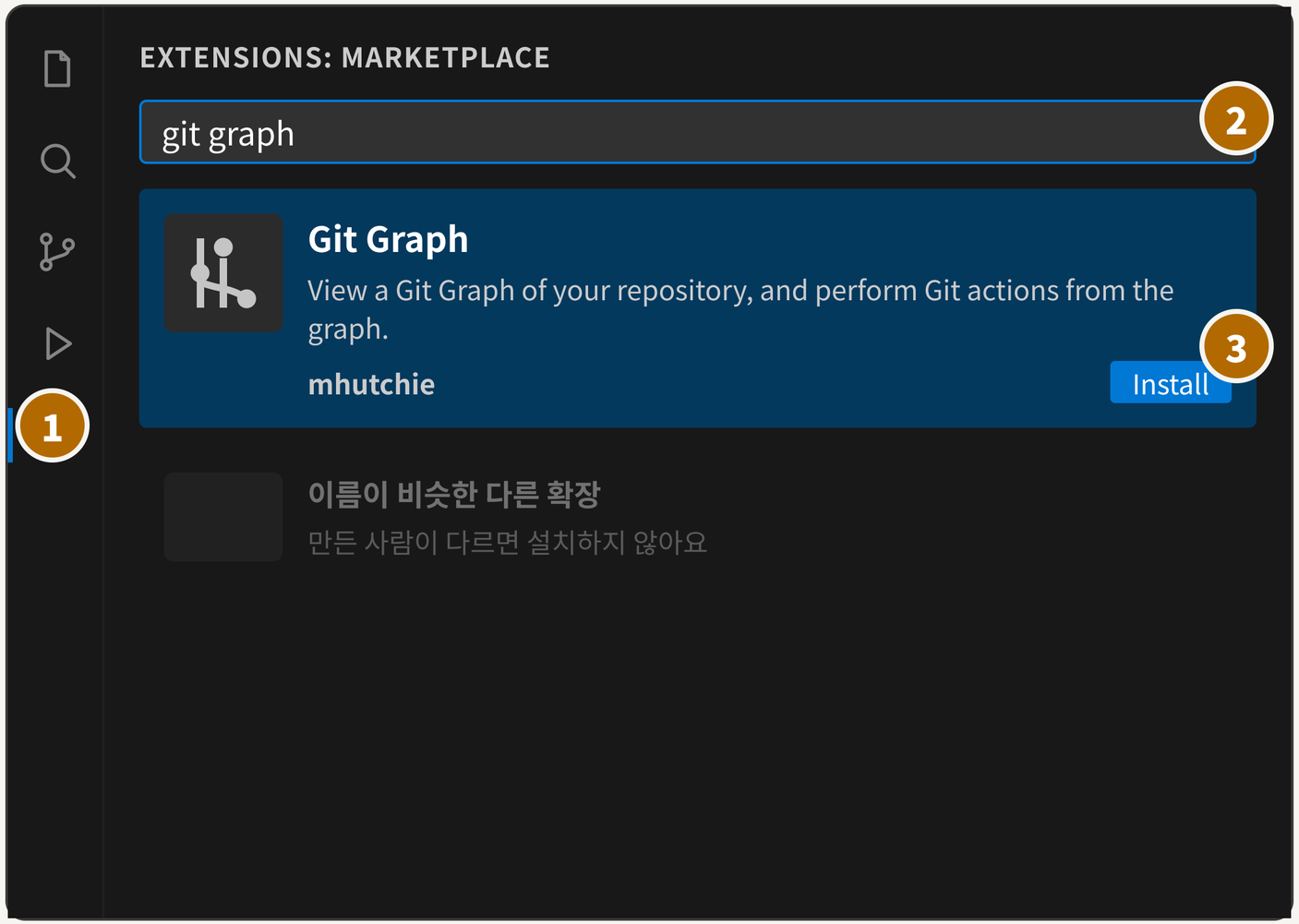


Graph	Description	Date	Author	Commit
	main origin Merge branch 'feature/login'	29 Oct 2026 10:06	kim	5991c366
	docs: README에 실행 방법 추가	29 Oct 2026 10:05	kim	ad50d357
	feature/login feat: 비밀번호 찾기 추가	29 Oct 2026 10:04	kim	dd7f15af
	docs: README 추가	29 Oct 2026 10:03	lee	b9f30e50
	feat: 로그인 버튼 추가	29 Oct 2026 10:02	kim	15d92cbb
	feat: 로그인 화면 추가	29 Oct 2026 10:01	kim	e12ffc2e

같은 기록이에요. 갈라진 길과 합쳐진 곳이 선으로 보여요.

설치하고 열기

확장 탭에서 Git Graph를 찾아 설치해요

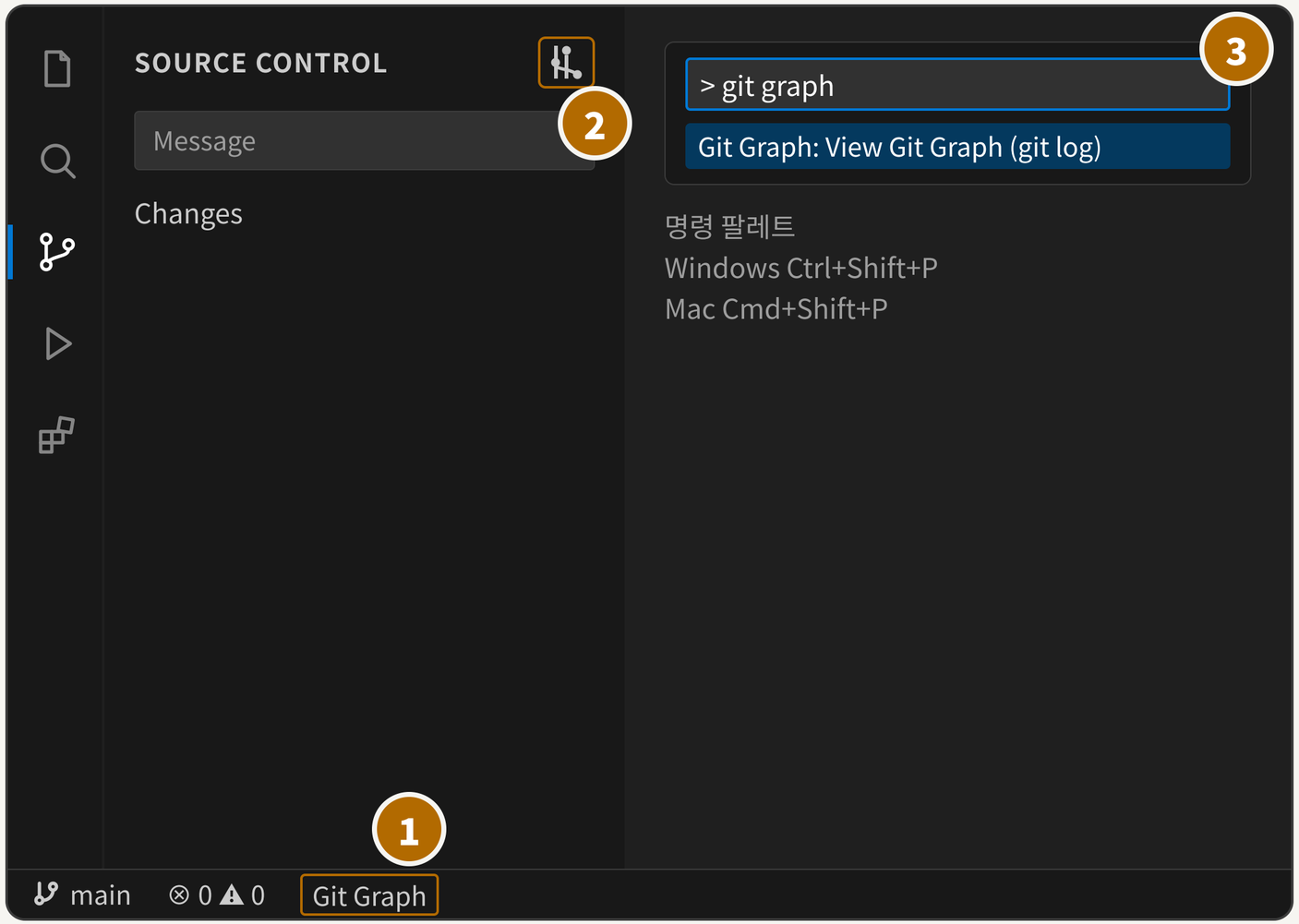


1. **왼쪽 막대의 확장 아이콘을 눌러요**
   Windows Ctrl+Shift+X
   Mac Cmd+Shift+X
2. **검색창에 git graph를 쳐요**
3. **만든 사람이 mhutchie인지 보고 Install을 눌러요**
   설치는 한 번만 하면 돼요

설치가 끝나면 Install 버튼 자리가 톱니바퀴로 바뀌어요.

설치하고 열기

저장소 폴더를 열고, Git Graph를 눌러요



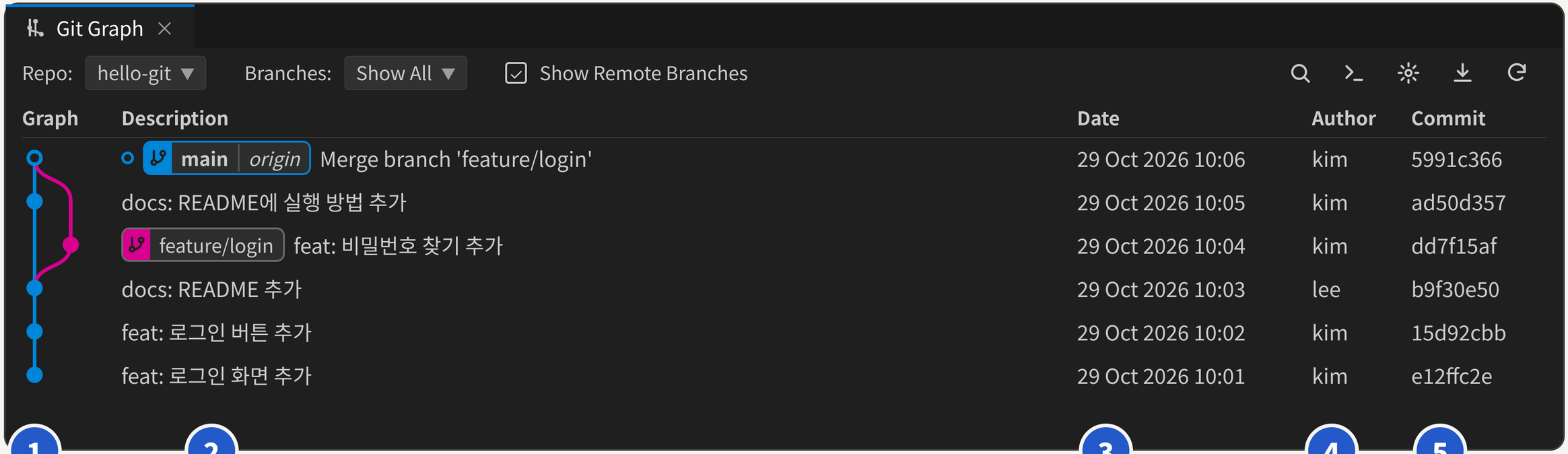
여는 법 세 가지

1. **아래 상태 표시줄의 Git Graph**
   가장 쉬워요
2. **소스 제어 탭 위의 그래프 아이콘**
   왼쪽 막대의 갈래 모양 아이콘
3. **명령 팔레트에서 git graph**
   View Git Graph (git log)를 골라요

먼저 File > Open Folder로
hello-git 폴더를 열어 둬요.

화면 읽기

한 줄이 커밋 하나예요



1 그래프
점과 선

2 설명
이름표와 커밋 메시지

3 날짜

4 작성자

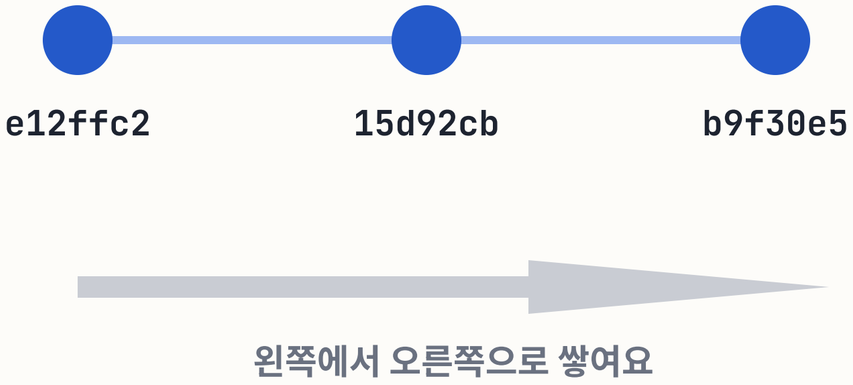
5 커밋 번호

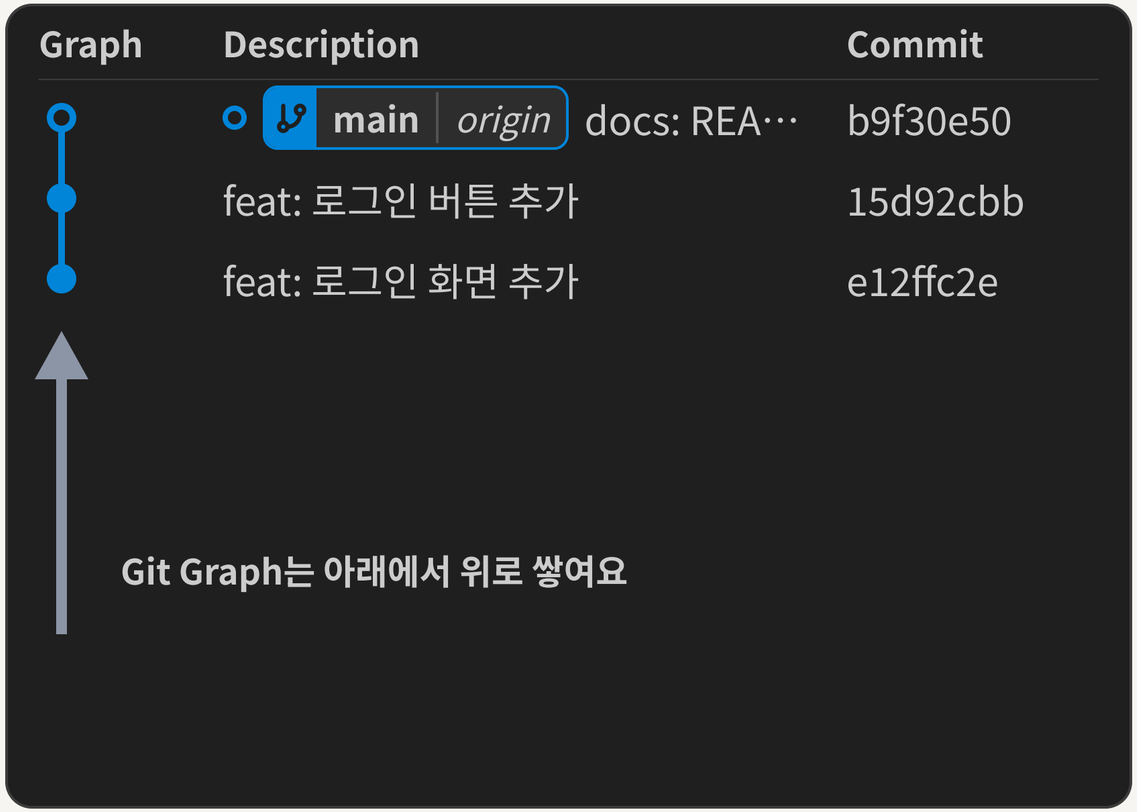
맨 위 줄의 Repo는 저장소 이름, Branches: Show All은 모든 브랜치를 보여 준다는 뜻이에요. 그대로 두면 돼요.

화면 읽기

03의 그래프를 세운 모양이에요

03 슬라이드의 그림

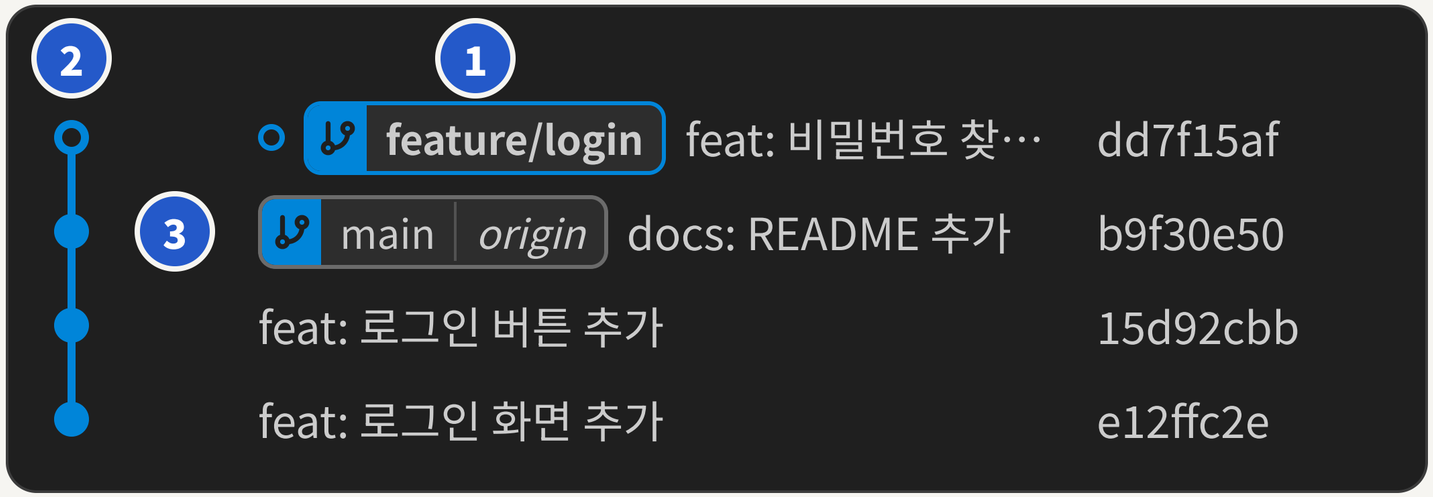




점 하나가 커밋 하나, 맨 위가 가장 최근이에요. 커밋 번호는 8자리로 나와요. 터미널의 7자리와 앞부분이 같아요.

화면 읽기

굵은 이름표가 지금 있는 브랜치예요



1 굵은 글씨, 색 테두리

지금 있는 브랜치예요.
터미널의 HEAD -> 와 같아요.

2 속이 빈 점

HEAD가 가리키는 커밋이에요.
지금 내 폴더가 이 모습이에요.

3 main | origin

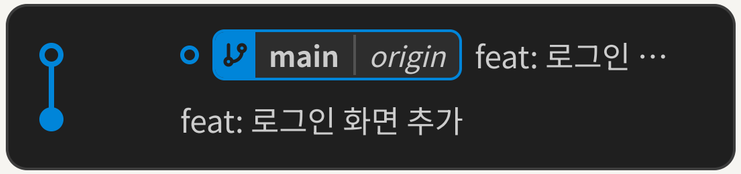
main과 origin/main이
같은 커밋에 있어요.

```
$ git log --oneline
dd7f15a (HEAD -> feature/login) feat: 비밀번호 찾기 추가
b9f30e5 (origin/main, main) docs: README 추가
```

화면 읽기

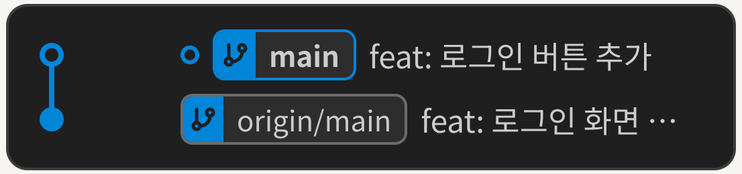
origin/main은 GitHub의 main이에요

같아요



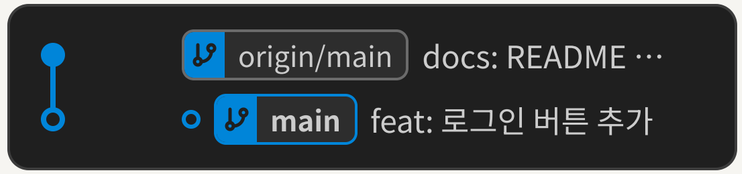
할 일 없음

내 컴퓨터가 앞서요



`git push origin main`

GitHub이 앞서요



`git pull origin main`

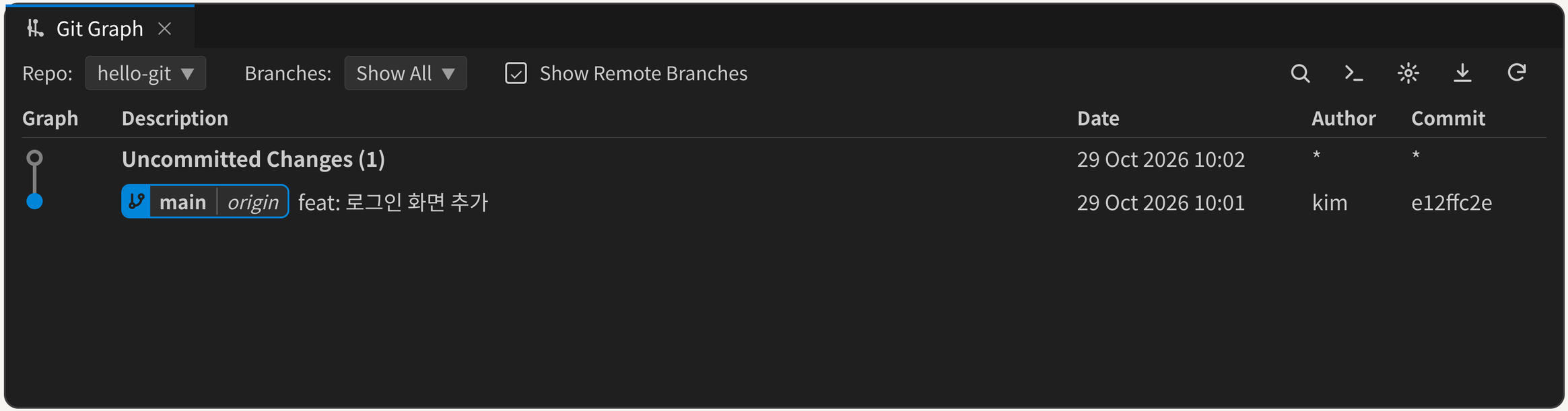
origin/main은 마지막으로 GitHub과 주고받았을 때의 main 위치예요.

팀원이 GitHub에 새로 올린 커밋은 git fetch origin을 쳐야 Git Graph에 나타나요.

02에서 git log --oneline과 git log --oneline origin/main을 따로 쳐서 비교했던 걸 한 화면에서 봐요.

실습 1 · 고치고 아직 커밋 전

echo "로그인 버튼" >> login.md



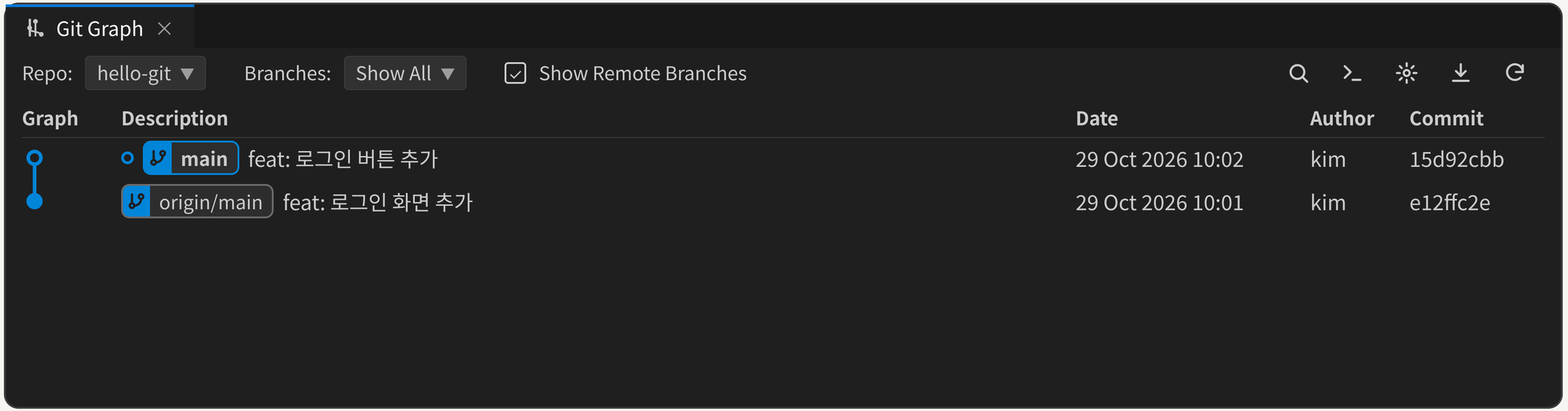

```
$ echo "로그인 버튼" >> login.md
$ git status
On branch main
Changes not staged for commit:
        modified:   login.md
```

맨 위에 회색 빈 점

Uncommitted Changes (1)
= 아직 커밋 안 한 파일 1개

실습 2 · 커밋하면

git commit -m "feat: 로그인 버튼 추가"



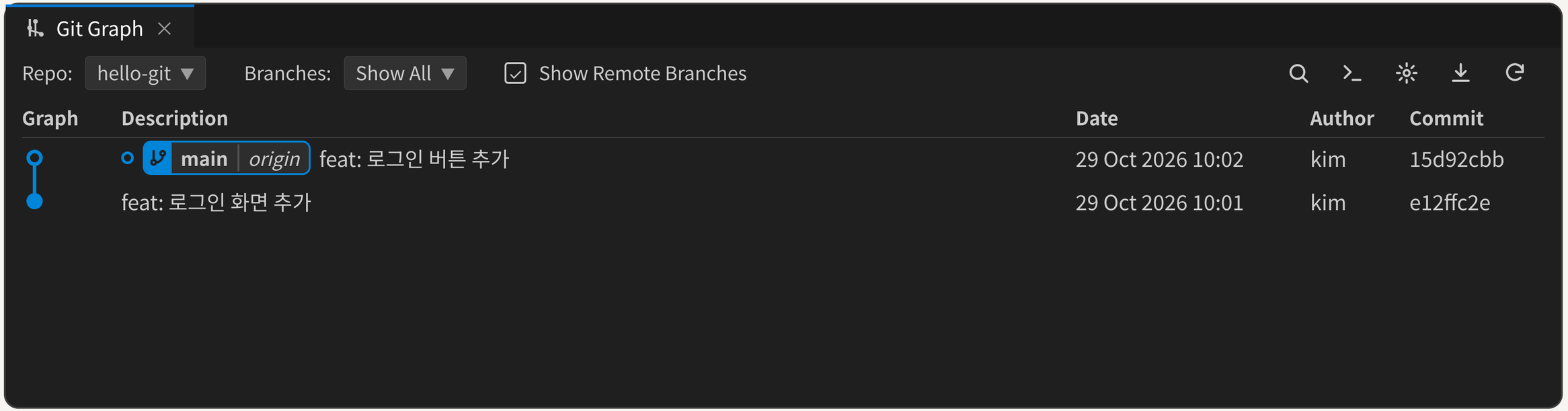

```
$ git add login.md
$ git commit -m "feat: 로그인 버튼 추가"
[main 15d92cb] feat: 로그인 버튼 추가
 1 file changed, 1 insertion(+)
```

새 점이 위에 생기고
main이 따라 올라가요

origin/main은 아직 아래에 있어요

실습 3 · push하면

git push origin main



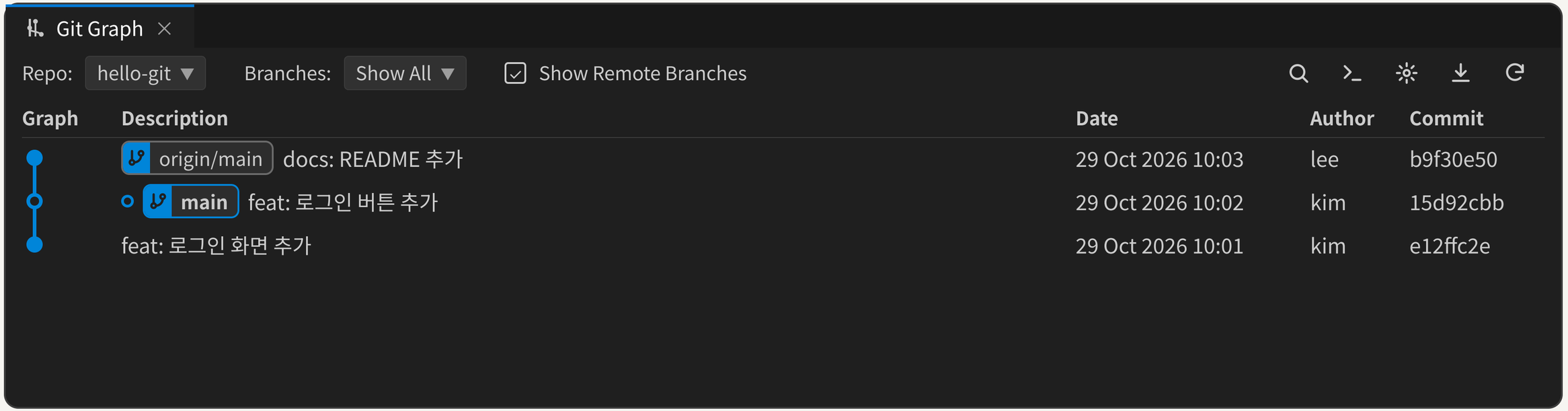

```
$ git push origin main
To https://github.com/kim/hello-git.git
   e12ffc2..15d92cb  main -> main
```

origin이 main 옆에 붙었어요

GitHub도 같은 커밋이에요

실습 4 · 팀원이 올린 걸 확인하면

git fetch origin



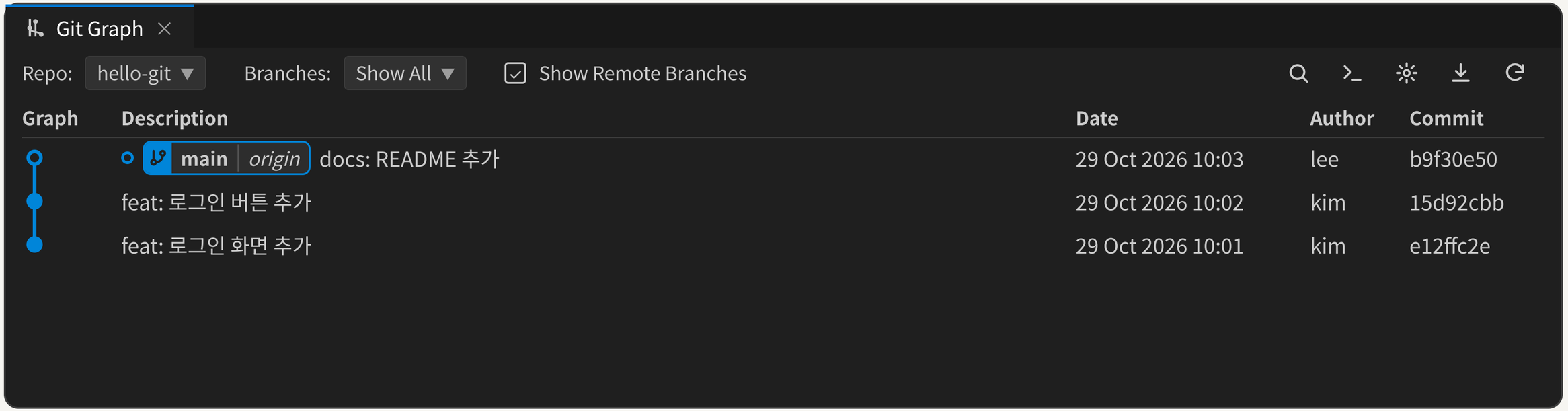

```
# 팀원 lee가 README.md를 만들어 GitHub에 push했어요
$ git fetch origin
From https://github.com/kim/hello-git
   15d92cb..b9f30e5  main       -> origin/main
```

origin/main이 main보다 위로

GitHub이 커밋 하나 앞서요

실습 5 · pull하면

git pull origin main



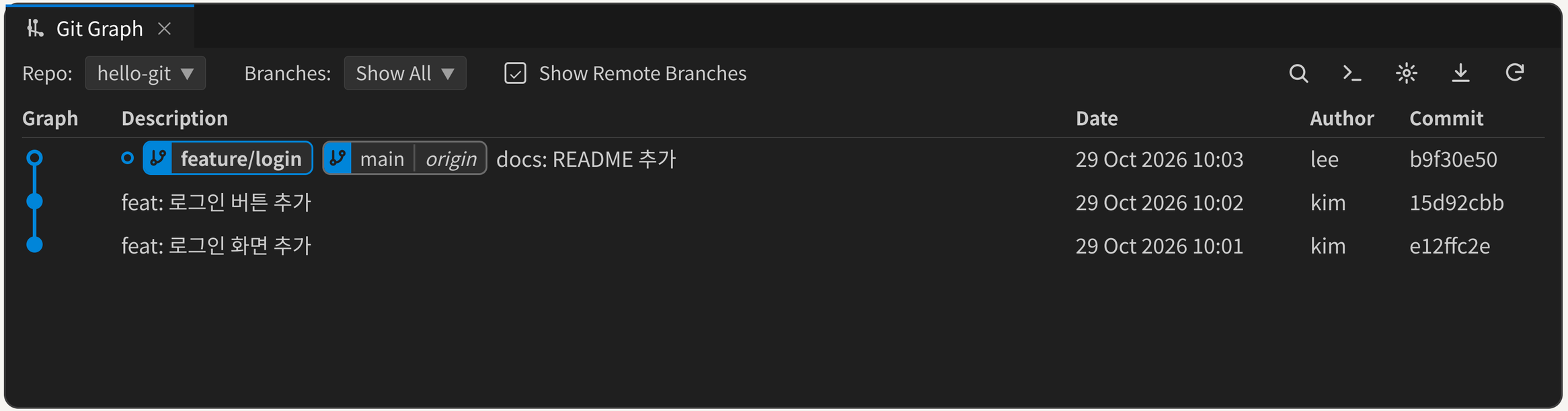

```
$ git pull origin main
From https://github.com/kim/hello-git
 * branch            main       -> FETCH_HEAD
Updating 15d92cb..b9f30e5
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

main이 올라가서 다시 붙었어요

내 폴더에 README.md가 생겨요

실습 6 · 브랜치를 만들면

git checkout -b feature/login



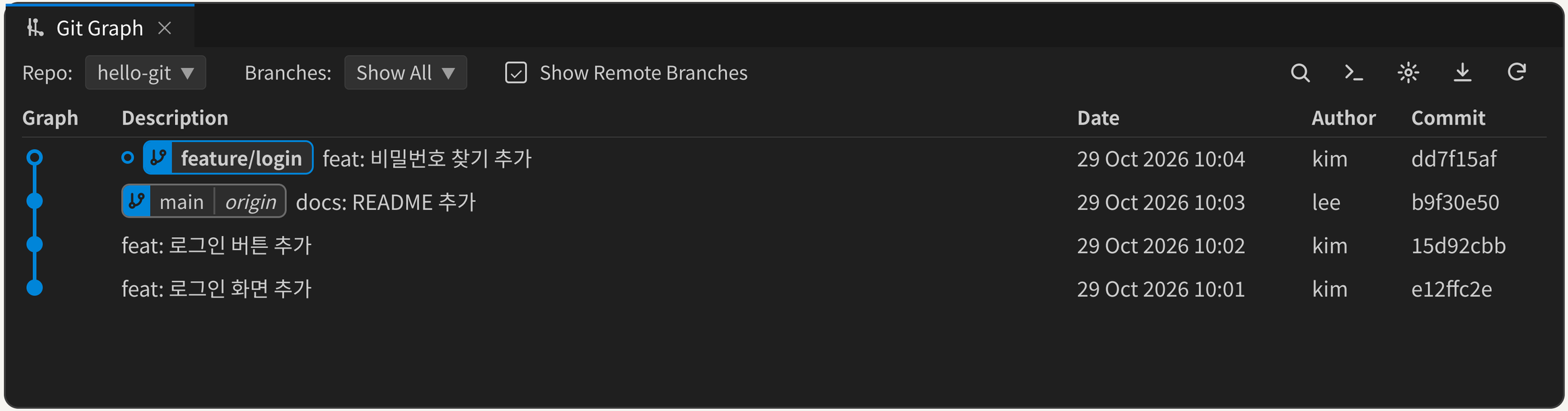

```
$ git checkout -b feature/login
Switched to a new branch 'feature/login'
$ git log --oneline
b9f30e5 (HEAD -> feature/login, origin/main, main) docs: README 추가
```

같은 점에
이름표가 하나 더

굵은 글씨가 feature/login으로

실습 7 · 브랜치에서 커밋하면

git commit -m "feat: 비밀번호 찾기 추가"



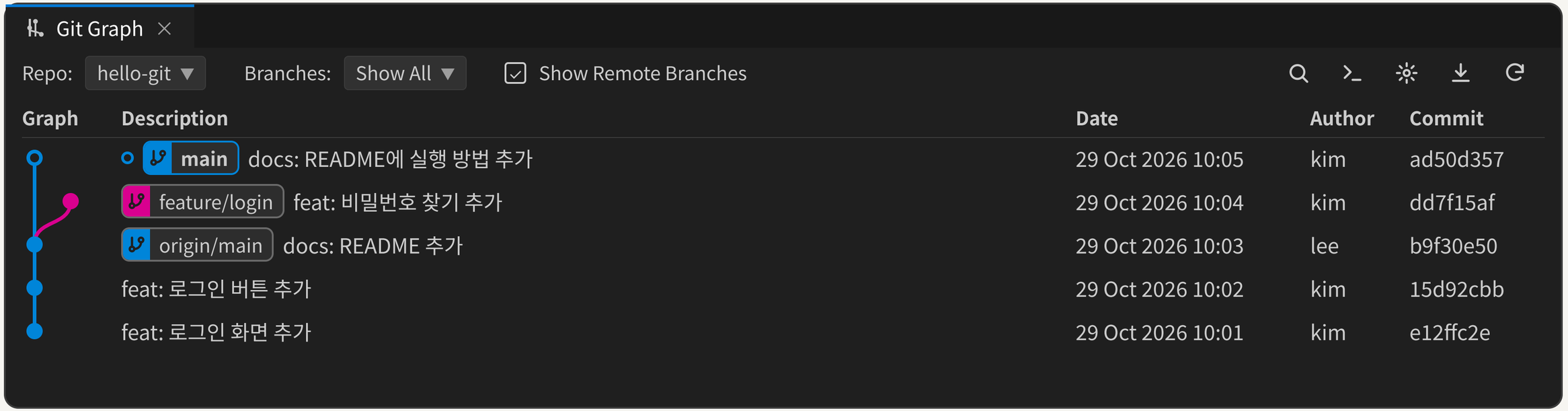

```
$ echo "비밀번호 찾기" >> login.md
$ git add login.md
$ git commit -m "feat: 비밀번호 찾기 추가"
[feature/login dd7f15a] feat: 비밀번호 찾기 추가
 1 file changed, 1 insertion(+)
```

feature/login만 위로 올라가요

main은 제자리예요

실습 8 · main으로 돌아가 커밋하면

git checkout main → git commit



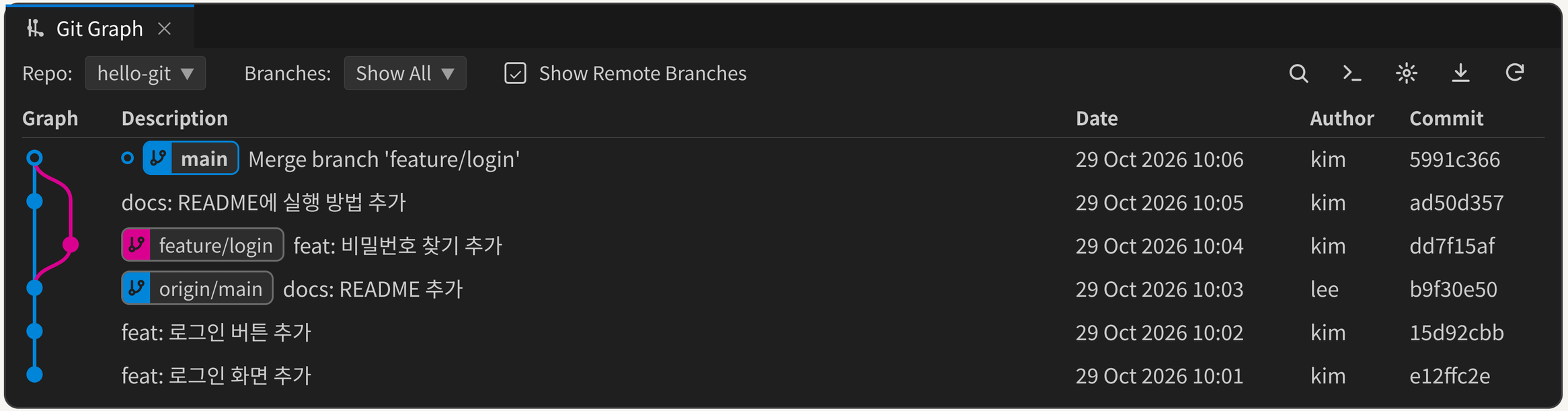

```
$ git checkout main
Switched to branch 'main'
$ echo "실행 방법은 준비 중이에요" >> README.md
$ git add README.md
$ git commit -m "docs: README에 실행 방법 추가"
[main ad50d35] docs: README에 실행 방법 추가
```

길이 둘로 갈라졌어요

분홍 선이 feature/login

실습 9 · merge하면

git merge feature/login



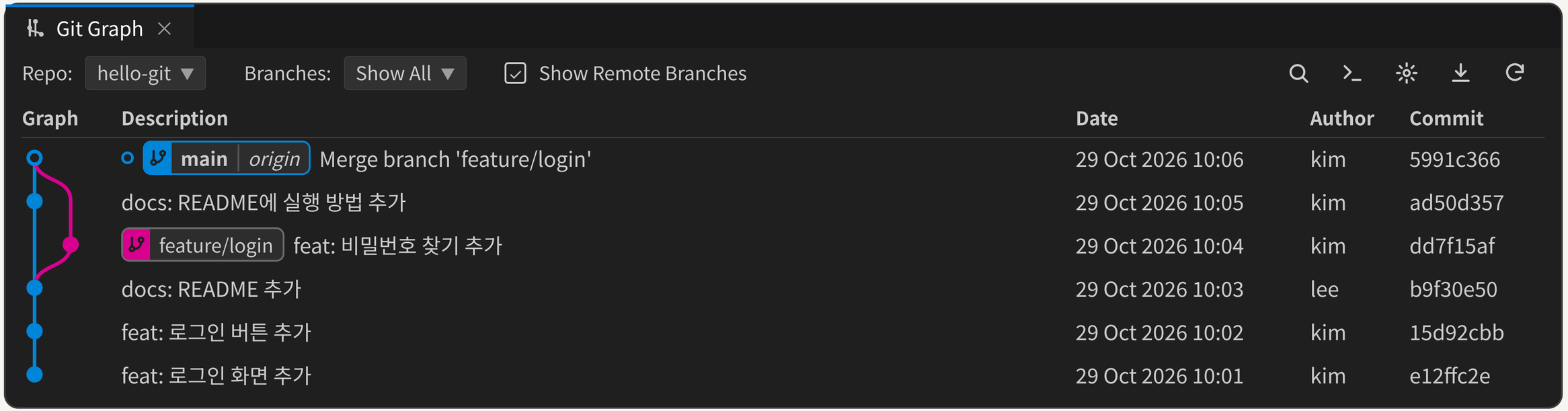

```
$ git merge feature/login
# vim이 열리면 Esc, :wq, Enter
Merge made by the 'ort' strategy.
 login.md | 1 +
 1 file changed, 1 insertion(+)
```

두 선이
merge 커밋에서 만나요

점 하나에 선 두 개

실습 10 · 다시 push하면

git push origin main




```
$ git push origin main
To https://github.com/kim/hello-git.git
   b9f30e5..5991c36  main -> main
```

main | origin이 맨 위에 같이

GitHub에도 merge가 올라갔어요

명령 치고 확인하기

커밋 줄을 누르면 바뀐 파일이 보여요

Graph	Description	Date	Author	Commit
	main origin Merge branch 'feature/login'	29 Oct 2026 10:06	kim	5991c366
	docs: README에 실행 방법 추가	29 Oct 2026 10:05	kim	ad50d357
	feature/login feat: 비밀번호 찾기 추가	29 Oct 2026 10:04	kim	dd7f15af
	docs: README 추가	29 Oct 2026 10:03	lee	b9f30e50
	feat: 로그인 버튼 추가	29 Oct 2026 10:02	kim	15d92cbb
	feat: 로그인 화면 추가	29 Oct 2026 10:01	kim	e12ffc2e

Commit: dd7f15afd87113c573611516bf2699f45cc00dd0
Parents: b9f30e506669024d7ccb5c55a2f8708b0e2d9db7
Author: kim <kim@example.com>
Committer: kim <kim@example.com>
Date: Thu Oct 29 2026 10:04:00 GMT+0900

feat: 비밀번호 찾기 추가

hello-git

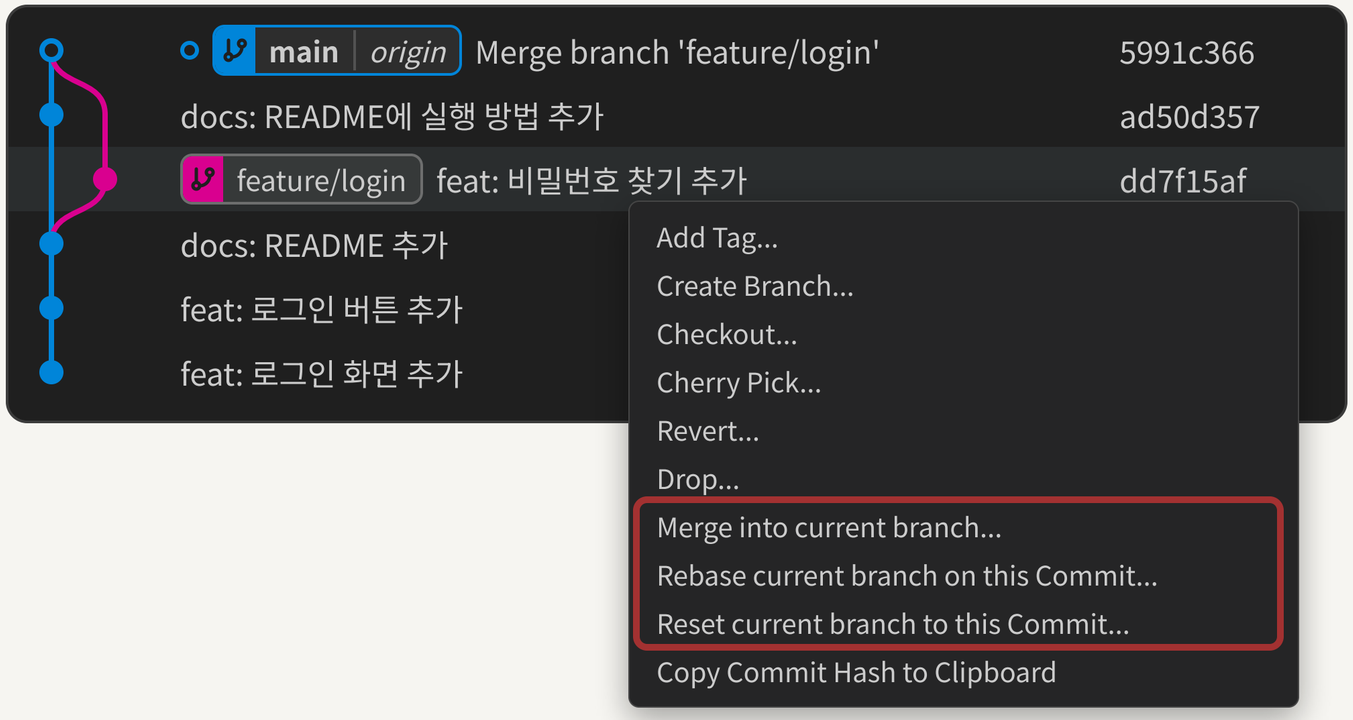
login.md (+1|-0)

파일 이름을 누르면 04에서 본 diff 화면이 열려요

터미널의 git show와 같은 내용이에요. 다시 누르면 닫혀요.

쓸 때 주의

명령은 터미널에서, 확인은 Git Graph에서



오른쪽 클릭 메뉴로 checkout, merge, reset도 할 수 있어요

수업에서는 메뉴를 쓰지 않아요. 무엇을 쳤는지 터미널에 남아야 오류가 났을 때 읽고 고칠 수 있어요.

오른쪽 위 Fetch 버튼 대신 git fetch origin을 쳐요.

터미널: 명령 **Git Graph: 확인**

막힐 때

Git Graph가 안 열리거나 그림이 그대로예요

1 Git Graph를 눌러도 저장소가 없다고 나와요

No Git repositories were found... 가 보이면 폴더를 잘못 연 거예요

```
File > Open Folder
hello-git 폴더 열기
```

2 팀원이 올린 커밋이 안 보여요

Git Graph는 내 컴퓨터에 받아 둔 기록만 보여 줘요

```
git fetch origin
```

3 명령을 쳤는데 그림이 그대로예요

보통 저절로 바뀌어요. 안 바뀌면 오른쪽 위 새로고침

```
↻ Refresh
```

한 장 요약

설치

확장 탭
Ctrl+Shift+X
git graph 검색
mhutchie
Install

열기

폴더 열기
hello-git

상태 표시줄의
Git Graph

읽기

점 = 커밋
맨 위 = 최신
굵은 이름표 = HEAD
빈 점 = 지금 커밋
main | origin
= GitHub과 같음

확인

명령은 터미널
결과는 Git Graph

팀원 커밋은
git fetch origin

명령을 칠 때마다 Git Graph를 한 번씩 보는 습관을 들여요.